

Plantilla multi-formato para investigación aplicada en buenas prácticas de desarrollo de software

Autor Uno*, Autor Dos†

*Afilación 1, Ciudad, País — autor1@ejemplo.edu

†Afilación 2, Ciudad, País — autor2@ejemplo.edu

Resumen—El Sistema de Control de Comparendos Policiales (SCMC) del municipio de Palermo, un proyecto académico desarrollado en C# .NET 8 y Angular 16+, se encuentra en una fase avanzada de definición de su lógica de negocio, habiendo estabilizado sus módulos de gestión de infracciones y acuerdos de pago. Sin embargo, la documentación de requisitos del sistema aún contempla un proceso de recaudo manual y presencial, el cual representa un cuello de botella operativo. Este artículo propone y justifica la integración de la pasarela de pago Wompi como la evolución técnica necesaria. Basado en el análisis de casos de estudio análogos (Artículo 10), se establece que el principal riesgo operacional en sistemas de recaudo es la aplicación manual diferida de pagos (24-48 horas). Proponemos una arquitectura de Integración Servidor-a-Servidor (S2S) mediante Webhooks, un modelo que garantiza la conciliación inmediata y la integridad de los datos. Esta implementación se alinea con la estrategia de "time-to-market" (Artículo 2) y es financieramente viable (Artículo 10). El éxito de la adopción ciudadana se supedita a los requisitos de Seguridad Percibida, exigiendo un frontend (Angular) con Responsive Web Design (RWD) y la visibilidad del ícono PSE (Artículo 16).

Index Terms—buenas prácticas, ingeniería de software, investigación aplicada, reproducibilidad

I. INTRODUCCIÓN

I-A. Contexto del Proyecto Académico SCMC

El SCMC es un proyecto académico de software integral diseñado para la Alcaldía de Palermo, enfocado en modernizar la gestión de comparendos del Código de Policía. El sistema, construido sobre un *stack* tecnológico robusto (C# .NET 8 y Angular 16), ya define una arquitectura avanzada que incluye autenticación JWT, gestión de roles (RBAC) y un diseño de base de datos modular.

I-B. El Problema: El Vacío Transaccional

A pesar de la avanzada arquitectura de gestión de datos, el SCMC aún no ha abordado la digitalización del módulo de pago. La especificación de requisitos actual contempla un proceso obsoleto: el ciudadano debe realizar los pagos ^{en} la Alcaldía Municipal o en los bancos autorizados gestionar los acuerdos "presencialmente".

Este modelo manual presenta tres problemas críticos que el SCMC busca resolver:

- **Ineficiencia Operativa:** Requiere conciliación manual de extractos bancarios, un proceso lento y propenso a errores.

- **Mala Experiencia Ciudadana:** El ciudadano no recibe la subsanación (paz y salvo) de su multa en tiempo real, generando desconfianza.
- **Costos Ocultos:** La gestión manual de recaudos incrementa la carga administrativa y los costos operativos, un problema documentado extensamente en la literatura (Artículo 10).

I-C. Propuesta y Objetivo

Este artículo propone la implementación de la pasarela de pago Wompi como el componente faltante para completar el ciclo de vida digital del comparendo. El objetivo es detallar la arquitectura técnica y la justificación estratégica para una implementación ágil S2S (Servidor-a-Servidor) que asegure la conciliación automática y en tiempo real de los comparendos y acuerdos de pago.

II. MARCOS TÉCNICOS Y ESTUDIOS RELACIONADOS (RELACIONADOS)

La decisión de proponer la integración de Wompi se fundamenta en un análisis de la literatura sobre implementaciones de e-commerce y recaudos masivos.

II-A. El Caso de la Conciliación Manual (Artículo 10)

El estudio de caso sobre la mejora en la aplicación de recaudos de matrículas en Uniminuto (Artículo 10) es el principal referente. Dicho estudio identificó que la aplicación manual diferida de pagos (24-48 horas) era el mayor punto de dolor del proceso. La solución propuesta fue la automatización mediante la integración del sistema financiero (ERP) con la PP a través de BAPIS y conectores. Esto demuestra que el problema que enfrenta el SCMC es un desafío de integración conocido y resuelto en el sector de recaudos.

II-B. Estrategia de Implementación: Agilidad vs. Construcción Propia (Artículo 2)

El caso de "Pasarela Virtual" (Artículo 2) analizó dos escenarios para la innovación tecnológica: desarrollo interno (Solución 1) versus tercerización (Solución 2). La tercerización (análoga a integrar Wompi) demostró ser drásticamente más rápida (8 meses vs. 18 meses para salir a producción). Para el SCMC, un proyecto académico enfocado en la agilidad, integrar Wompi es la estrategia de *time-to-market* más eficiente.

II-C. Requerimientos de Adopción Ciudadana (Artículo 16)

Un sistema público no solo debe funcionar, debe ser usado. El estudio UTAUT2 (Artículo 16) sobre la adopción de pagos en línea identificó los factores no técnicos que determinan el éxito:

- **Confianza y Seguridad Percibida:** Son las variables más influyentes.
- **Señales de Confianza Específicas:** La confianza en Colombia no se genera con íconos genéricos, sino con sellos locales. El ícono PSE es el factor de confianza más crítico.
- **Diseño (RWD):** El Responsive Web Design (que el sitio se adapte al móvil) tiene un efecto estadísticamente significativo en la intención de completar el pago.

II-D. El Modelo Arquitectónico (Artículo 19)

El proyecto Çarwash”(Artículo 19) documenta un *stack* tecnológico similar (Java/Angular, Microservicios, MySQL). Su arquitectura proporciona un modelo validado para el SCMC, utilizando un API Gateway para gestionar las comunicaciones y servicios en la nube como AWS SNS/SES para las notificaciones automáticas de pago, un patrón que el SCMC (C#) puede replicar.

II-E. Requisitos de Seguridad Obligatorios (Artículo 20)

La integración de una PP es una obligación regulatoria. En Colombia, la Superfinanciera exige el cumplimiento de la norma PCI DSS 3.2.1 desde 2018. Al utilizar una PP certificada como Wompi, el SCMC delega la complejidad de esta certificación (firewalls, cifrado de datos, monitoreo de seguridad), reduciendo drásticamente el riesgo y el costo del proyecto.

III. METODOLOGÍA

La metodología para esta implementación es Propositiva-Técnica, basada en los hallazgos de los estudios relacionados y el estado actual del SCMC.

Fase de Diagnóstico (DMAIC - Definir): Se define el problema (Artículo 10) como la latencia en la conciliación manual. Se mide el proceso actual (documentado en los requisitos del SCMC) que depende de pagos presenciales en bancos.

Fase de Diseño de Solución (DMAIC - Mejorar): Se diseña la arquitectura S2S (Servidor-a-Servidor) con Webhooks como la solución técnica. Esto se alinea con el trabajo ya realizado en el SCMC (rutas y migraciones), que prepara al backend C# para recibir estas notificaciones.

Fase de Justificación (Análisis Financiero): Se utiliza el modelo financiero del Artículo 10 (VPN, TIR, B/C) para justificar la inversión en horas de desarrollo (C#/Angular) y licenciamiento de Wompi, demostrando un ROI positivo a través del ahorro en costos operativos.

Definición de Requerimientos No Funcionales: Se establecen los requisitos de UX/UI (RWD, ícono PSE) basados en la evidencia de Artículo 16, y los requisitos de seguridad (PCI DSS) basados en Artículo 20.

IV. IMPLEMENTACIÓN (EL MODELO WOMPI CON C# Y ANGULAR)

La implementación se divide en el frontend (solicitud) y el backend (conciliación), aprovechando el *stack* tecnológico existente del SCMC.

IV-A. Frontend (Angular): La Puerta de Confianza

El rol de Angular es iniciar el pago y generar confianza.

Interfaz de Pago: El componente de Angular debe ser 100 % Responsive (RWD), adaptándose a móviles y computadoras.

Señales de Confianza: Debe mostrar prominentemente los logos de Wompi y el ícono PSE (Artículo 16) para asegurar la percepción de seguridad.

Flujo de Redirección:

- El ciudadano confirma la multa o acuerdo de pago en Angular.
- Angular llama al backend C# (Endpoint 1) para obtener los datos de la transacción.
- C# devuelve la URL de *checkout* de Wompi.
- Angular redirige al ciudadano a la página segura de Wompi (Checkout por Redireccionamiento).

IV-B. Backend (C#): El Motor de Conciliación

El backend de C# es el cerebro de la operación. Debe exponer dos *endpoints* (Controladores) críticos:

Endpoint 1: C# - Controlador de Inicio de Pago ([HttpPost] /api/pagos/iniciar)

- **Función:** Recibe la solicitud de Angular (ej. MultaID, Monto).
- **Lógica C#:**
 1. Consulta la multa en la DB (SQL Server, PostgreSQL o MySQL).
 2. Genera una Referencia Única de transacción para el SCMC.
 3. Calcula la Firma de Integridad (signature - Hash SHA-256) usando la llave privada de Wompi. Esto es esencial para que Wompi acepte la transacción (Artículo 20).
 4. Define la `redirect_url` (a dónde vuelve el usuario) y, crucialmente, la `webhook_url` (Endpoint 2).
 5. Devuelve el objeto de solicitud (o la `checkoutUrl`) a Angular.

Endpoint 2: C# - Controlador de Webhook (CRÍTICO) ([HttpPost] /api/webhooks/wompi-callback)

- **Función:** Este *endpoint* no es llamado por el usuario, sino por el servidor de Wompi de forma asíncrona (S2S) cuando el pago es APROBADO.
- **Lógica C# (Aplicación Atómica):**

1. **Verificación de Seguridad:** El código C# debe validar la firma de integridad del Webhook para asegurar que la solicitud es auténtica y no fraudulenta (Artículo 20).
2. **Conciliación:** Si la firma es válida y el estado es APROBADO, el backend de C# ejecuta una transacción de base de datos atómica: `UPDATE Multas SET Estado = 'Pagada', IDTransaccionWompi = @id_wompi WHERE Referencia = @ref_wompi.`
3. **Notificación:** Dispara el servicio de notificaciones (Email/Telegram) para enviar el Paz y Salvo por email/SMS al ciudadano.

V. RESULTADOS (PROYECTADOS)

La adopción de este modelo en el SCMC generará resultados inmediatos y cuantificables.

Resultados Operacionales: El tiempo de aplicación de pagos se reduce de 24-48 horas a menos de 1 minuto. Se eliminan los errores manuales de digitalización y se optimiza la generación de reportes para la inspección y finanzas.

Resultados Financieros (Proyectados por Artículo 10):

- La inversión en el desarrollo (C#/Angular) y licenciamiento se justifica con un TIR proyectado del 44 % y una relación Beneficio/Costo de 1.38.
- Reducción de Gastos Administrativos: Se proyecta un ahorro de hasta el 50 % en costos de personal de recaudo.

Resultados de Adopción (UX): La implementación de RWD y el ícono PSE en Angular (Artículo 16) maximizará la tasa de conversión de pagos, asegurando una alta adopción ciudadana.

VI. DISCUSIÓN

El SCMC se encuentra en la Fase 3 (Procesamiento) de la implementación de e-commerce (Artículo 12). El análisis de la literatura confirma que la decisión de implementar Wompi es acertada.

La discusión central es que el verdadero trabajo de integración no ocurre en el frontend de Angular (que solo dirige), sino en el backend de C#. El Controlador de Webhook es el componente más crítico del sistema transaccional.

El trabajo reciente del equipo de desarrollo (migraciones de DB y ajuste de rutas) es fundamental, ya que este *endpoint* de Webhook debe ser estable, seguro y rápido. Si el backend C# no procesa el Webhook de forma atómica e instantánea, el proyecto fracasará en su objetivo principal y volverá al problema de la latencia (el problema de Artículo 10). La integración de la PP consolida la robusta arquitectura del SCMC, que ya incluye patrones de diseño avanzados como Strategy y Repository, y un sistema de seguridad granular basado en roles y JWT.

VII. CONCLUSIONES

Implementación Obligatoria: La integración de una PP es un requisito no negociable para la viabilidad del SCMC. Basado en el análisis de mercado, Wompi es una elección estratégica y robusta.

Arquitectura S2S/Webhook: El modelo de Checkout por Redireccionamiento + Webhooks S2S es la arquitectura óptima. Es la más rápida de implementar (Artículo 2) y la única que garantiza la conciliación en tiempo real (Artículo 10).

Responsabilidades del Stack:

- **C# (Backend):** Debe ser el guardián de la transacción. Su rol es validar criptográficamente el Webhook y aplicar el pago atómicamente en la DB.
- **Angular (Frontend):** Debe ser el motor de la confianza, implementando RWD y el ícono PSE para asegurar la máxima adopción ciudadana.

VIII. APÉNDICES

A. Flujograma S2S (C# / Angular): Diagrama de secuencia detallando: (1) Solicitud Angular ->C#, (2) Redirección a Wompi, (3) Webhook S2S (Wompi ->C#), (4) Aplicación Atómica C# ->DB, (5) Notificación al Ciudadano.

B. Arquitectura Lógica de Integración S2S: Diagrama que muestra el Microservicio de Integración y los flujos de comunicación (APIs de consulta, Webhooks de retorno).

C. Modelo Financiero (Adaptado de Artículo 10): Tablas de VPN, TIR, y B/C que justifican la inversión en el desarrollo de la integración del SCMC.

D. Checklist de UX/UI (Basado en Artículo 16): Checklist de implementación en Angular para RWD y visibilidad del ícono PSE.

E. Requerimientos de Seguridad (Artículo 20): Resumen de los controles obligatorios de PCI DSS 3.2.1 que son delegados a Wompi.

REFERENCIAS

- [1] M. Fowler y M. Foemmel, «Continuous Integration,» en *ThoughtWorks*, 2006. dirección: <https://martinfowler.com/articles/continuousIntegration.html>
- [2] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley Professional, 2003, ISBN: 0321146530.
- [3] L. Chen, «Continuous Delivery: Huge Benefits, but Challenges Too,» en *IEEE Software*, vol. 32, 2015, págs. 50-54. DOI: 10.1109/MS.2015.27
- [4] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008, ISBN: 9780132350884.
- [5] N. Forsgren, «DORA Metrics in Practice,» *ACM Queue*, 2021. dirección: <https://queue.acm.org/>
- [6] N. Forsgren, J. Humble y G. Kim, «Accelerate: The Science of Lean Software and DevOps,» *IT Revolution*, 2018.

- [7] G. Fitzpatrick y M.-A. Storey, «The Risks of Good Enough Software Engineering,» *IEEE Software*, vol. 34, n.º 6, págs. 14-19, 2017. doi: 10.1109/MS.2017.4121224